University of Mannheim
FACULTY OF BUSINESS INFORMATICS

_Club's information@cref _Player@cref _Player@cref

Group Project

Generation and querying of Time-aware Knowledge Graphs

Supervision: Prof. Dr. Heiko Paulheim PhD Researcher.Franz Krause
Student: Name: Mariam Marzouk ID: 1754976

GitHub project

Contents

1 Ontology development **3**
1.1 Class Creation . . . 3
1.2 Property creation . . . 4
1.2.1 Object property . . . 4
1.2.2 Data Property . . . 4
1.3 Individuals Declaration . . . 5
1.3.1 Cellfie plugin . . . 5

2 Ontology development methods **7**
2.1 Protégé . . . 7
2.2 Python . . . 7

3 Knowledge Graph **8**

4 Representing Temporality in Graphs **9**
4.1 Reification . . . 9

5 External Source **11**
5.1 Metadata . . . 11
5.2 Temporal data . . . 11

6 Interactive Prototype **13**

1 Ontology development

Ontologies are semantic data models that define the kind of things that exist in our domain and the properties that can be used to describe them. Ontologies are generalized data models, i.e., they model only general types of things that have certain properties, but don't contain information about specific individuals in our domain.[5]enterprise-knowledge Ontologies can capture highly complex ideas and business logic, provide more intuitive ways to structure information, and can ultimately enable new use cases such as semantic search, recommendation engines, and AI.

We chose **Sports domain** for developing our ontology and I worked on the **Basketball** Ontology. An ontology consists of three main components, usually described as follows: **Classes**: the distinct types of things that exist in our data. **Relationships**: Properties that link two classes together. **Attributes** : Properties that describe an individual class.

1.1 Class Creation

Classes are used to group entities with similar characteristics or properties. Classes are used to represent the concepts or types of things in the domain being modeled.
Classes can have sub-classes, which inherit the attributes and properties of the parent class. For example, a subclass of "Player" might be "Basketball_player", which would inherit the attributes of player " class, but would also have additional attributes such as "NBA ID".
Classes are a fundamental concept in ontology and knowledge graph, providing a way to structure and organize the entities and relationships in the knowledge representation. Those are the classes created for building Basketball Ontology:

Table 1: bbkg Classes a

Classes
Player
Sports_Club
Sports_League
Division
Conference
Position
Event
RealGM

Those are the classes created for the data exported from RealGM website.

Table 2: bbkg Classes and sub-classes for scrapped data from RealGM

Classes	Sub-classes
Award	
Player	Active_Player
	Inactive_Player

1.2 Property creation

"Property" in OWL is divided into "Object Property" and "Data Property". Properties have inheritance. If class A is connected with class B by Property P, all the sub-classes of A are connected with B by Property P.

1.2.1 Object property

An object property is a property that links two entities together. It represents a relationship between entities that can be described as a verb. For example, in the bbkg knowledge graph , an object property is "plays_for", linking a Player to a Sport_Club entity. Object properties can be transitive, symmetric, and reflexive.
In the basketball Ontology there are **9 Object properties / Relationships**.

Table 3: bbkg Object Properties

Object property	Domain	Range
plays_for	Player	Sports_Club
plays_as	Player	Position
participated_in	Player	Event
is_awarded	Player	Award
ID	Player	RealGM
competes_in	Sports_Club	Sports_League
based_in	Sports_Club	Country
part_of	Sports_Club	Division
is_in	Division	Conference

1.2.2 Data Property

Data property is a property that links an entity to a value, such as a string, number, or date. It represents a characteristic of the entity that can be described as an adjective. For example, in the bbkg knowledge graph representing a Player, a data property is "ID", linking a Player entity to a numerical value representing his ID number in RealGM website.
In the basketball Ontology there are **2 Data properties** created which are:

Table 4: bbkg Data Properties

Object property	Domain	Range
full_name	Player	xsd:string
conference	Player	xsd:string

1.3 Individuals Declaration

The individual should be the smallest object in ontology, which means that it is atomic and cannot be separated. If an individual can be divided by a smaller one, it should not be an individual but a class.
Creating individuals is an important step in building an ontology or knowledge graph, as it allows us to represent specific entities and their relationships within the domain being modelled.
By creating individuals, we can also populate the knowledge graph with data and make it more useful for reasoning and querying.
Ontology + Instances = knowledge graph
Using SPARQL query a csv file with all the data needed from Wikidata were exported, like Player, Sports Club, League, Position and Event, Sports Club Start date , Sports Club End date, Division and Conference, filtering only the Players who have NBA_ID to limit the size of the data.

	Player	RealgmID	Team_country	FullName	Team	Start	End	
0	http://www.wikidata.org/entity/Q963977	156	http://www.wikidata.org/entity/Q30	Dijon Thompson	http://www.wikidata.org/entity/Q796205	NaN	NaN	ht
1	http://www.wikidata.org/entity/Q963977	156	http://www.wikidata.org/entity/Q30	Dijon Thompson	http://www.wikidata.org/entity/Q46005	NaN	NaN	ht
2	http://www.wikidata.org/entity/Q963977	156	http://www.wikidata.org/entity/Q30	Dijon Thompson	http://www.wikidata.org/entity/Q643795	NaN	NaN	ht
3	http://www.wikidata.org/entity/Q963977	156	http://www.wikidata.org/entity/Q30	Dijon Thompson	http://www.wikidata.org/entity/Q164177	NaN	NaN	ht
4	http://www.wikidata.org/entity/Q963977	156	http://www.wikidata.org/entity/Q30	Dijon Thompson	http://www.wikidata.org/entity/Q3615392	2001-01-01T00:00:00Z	2005-01-01T00:00:00Z	ht

Figure 1: Wikidata extraction

1.3.1 Cellfie plugin

It is Protégé plugin for creating OWL ontologies from spreadsheets. I wanted to add my individual/ instances to my ontology in protégé, and Cellfie plugin was the solution.
First, I downloaded the plugin then in protégé I chose Tools – Create axioms from excel workbook then I uploaded the excel sheet that has the Instances. Then I created a transformation rule to be able to map rows and columns to I instances For the given example I am stating that the data starts from Column A till J and the start reading from row 2 till 100001 And added the rule that Class: Basketball_player = to @D2 which is DijonThompson @D2 is a MappingMaster which is a domain-specific language (DSL) that defines mappings from spreadsheet content to OWL ontologies. This language is based on the Manchester OWL Syntax, which is

itself a DSL for describing OWL ontologies. So, Instance Dijon Thompson was added under Basketball_Player But I added the individuals in python as it was more efficient an easier to pass through the whole csv file. [1] [2]

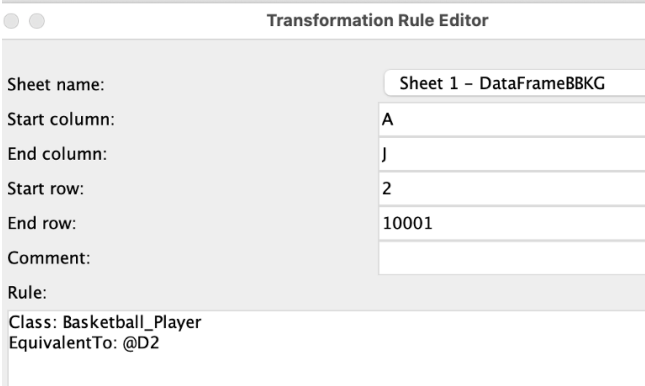

Figure 2: MappingMasterDSL

2 Ontology development methods

Building my ontology using both Protégé and RDF

2.1 Protégé

Protégé is an ontology editing tool developed by Stanford University. It is open-source software and has both an application-specific graphical user interface and an API interface, which greatly increases its flexibility. Protégé is one of the most common tools for editing ontologies [6].
In Protégé, "owl: thing" is the root of all the class. I have been using Protégé desktop version 5.5.0 I have built an Ontology **bb.owl** in Protégé however after different trials to load the file to add instance in Python it didn't work.

2.2 Python

Using RDFLIb library. RDFlib is widely used in the Semantic Web community and has a large ecosystem of plugins and extensions. It supports various RDF serialization formats, including RDF/XML, N-Triples, Turtle, and JSON-LD. Additionally, RDFlib has a flexible API that allows developers to work with RDF data at different levels of abstraction, from low-level triple manipulation to high-level querying and reasoning.

This data will be entered into a graph as a series of subject-predicate-object triples, where each triple represents a node (the subject) and its relation (the predicate) to another node (the object). RDF graphs use Uniform Resource Identifiers (URIs) to uniquely identify nodes and relationships, although an object can also be a literal value. [4] Building an ontology using RDFlib involves several steps:
1. Define the classes and properties.
2. Define the relationships between classes and properties.
3. Create instances of classes.
4. Query the ontology

RDFlib is widely used in the Semantic Web community and has a large ecosystem of plugins and extensions. It supports various RDF serialization formats, including RDF/XML, N-Triples, Turtle, and JSON-LD. Additionally, RDFlib has a flexible API that allows developers to work with RDF data at different levels of abstraction, from low-level triple manipulation to high-level querying and reasoning.

3 Knowledge Graph

A Knowledge graph is a directed, labeled graph in which domain-specific meanings are associated with nodes and edges. A node can represent any real-world entity, and an edge label captures the relationship between the two nodes of interest. Knowledge graph is a powerful tool for knowledge representation, information integration, and data analysis. Entities are represented as nodes and relationships between entities are represented as edges. This structure allows for a flexible and efficient way to represent complex information and answer complex queries. Knowledge graphs have many methods and applications.
One of them is that knowledge graph can be a powerful tool for integrating data from multiple sources, even if the data is stored in different formats or languages. By connecting data across different domains, knowledge graphs can enable organizations to gain new insights and make more informed decisions. [3] That method was what we were applying in the project, integrating multiple sources Wikidata and RealGM, so it can enable users with new insights.

Here are how the data of the player were shown: LeBron James of type Player and Active Player has **ID** , **contract** , **full_name**, **participated_in**, **play_as** and **plays_for** . Plays has ID which is the URL of the player in RealGM, Contract which is Literal of type string , full name which is Literal of type string, Participated_in Events, plays_in Position and plays_for a team.

```
<http://www.wikidata.org/entity/Q36159> a bbkg:Active_Player,
        bbkg:Player ;
    bbkg:ID <https://basketball.realgm.com/player/LeBron-James/summar
y/250> ;
    bbkg:contract "Under Contract (Los Angeles Lakers)" ;
    bbkg:full_name "LeBron James" ;
    bbkg:participated_in <http://www.wikidata.org/entity/Q178595>,
        <http://www.wikidata.org/entity/Q241329>,
        <http://www.wikidata.org/entity/Q8558>,
        <http://www.wikidata.org/entity/Q8567>,
        <http://www.wikidata.org/entity/Q8577> ;
    bbkg:play_as <http://www.wikidata.org/entity/Q1607137>,
        <http://www.wikidata.org/entity/Q212413>,
        <http://www.wikidata.org/entity/Q308879>,
        <http://www.wikidata.org/entity/Q462471> .
```

Figure 3: Player's information

```
[] a rdf:Statement ;
    bbkg:end "2018-01-01T00:00:00+00:00"^^xsd:dateTime ;
    bbkg:start "2014-01-01T00:00:00+00:00"^^xsd:dateTime ;
    rdf:object <http://www.wikidata.org/entity/Q162990> ;
    rdf:predicate bbkg:plays_for ;
    rdf:subject <http://www.wikidata.org/entity/Q36159> .
```

Figure 4: Player's information

Figure 5 shows the information of the Sports_Club which includes **based_in** (Sports_Club based in a country) , **competes_in** (Sports_Club competes in a League) and **part_of** (Sports_Club is part_of a division).

Figure 6 shows the information of the Division includes **is_in** Conference (Division is_in a Conference).

```
<http://www.wikidata.org/entity/Q193753> a bbkg:Sports_Club ;
    bbkg:based_in <http://www.wikidata.org/entity/Q30> ;
    bbkg:competes_in <http://www.wikidata.org/entity/Q1215884> ;
    bbkg:part_of <http://www.wikidata.org/entity/Q431925> .
```

Figure 5: Sports_Club's information

```
<http://www.wikidata.org/entity/Q431925> a bbkg:Division ;
    bbkg:is_in <http://www.wikidata.org/entity/Q276530> .
```

Figure 6: Division's information

4 Representing Temporality in Graphs

Temporal data refers to information that is associated with a specific period or timeline.

4.1 Reification

Reification is a technique used in knowledge graph modeling to add additional information about statements in the graph. The basic idea behind reification is to treat a statement (i.e., a triple consisting of subject, predicate, and object) as an entity in its own right, which can then have additional attributes and relationships associated with it.

The benefits of reification in knowledge graph modeling include the ability to capture more complex relationships between entities, including temporal and contextual information. For example, by reifying a statement, it is possible to add metadata about when the statement was made, who made it, and under what conditions it was made. This can be useful for analyzing and reasoning about the data, especially when dealing with large and complex data sets.

Reification provides a higher expressive power that permits one to express statement-level metadata (such as the validity of a statement). Re- lying on existing methods in Resource Descriptive Framework (RDF), standard reification introduces a blank node to notify an temporal statement and attaches other metadata into the blank node.

Figure 3: shows how reification was used in bbkg to extend <Player> <bbkg:plays_for> <Sports_Club> with start and end using blank node.

```
[] a rdf:Statement ;
    bbkg:end "2010-01-01T00:00:00+00:00"^^xsd:dateTime ;
    bbkg:start "2003-01-01T00:00:00+00:00"^^xsd:dateTime ;
    rdf:object <http://www.wikidata.org/entity/Q162990> ;
    rdf:predicate bbkg:plays_for ;
    rdf:subject <http://www.wikidata.org/entity/Q36159> .
```

Figure 7: Reification

Table 5: Reification table

Blank node	rdf:	Statement
Blank node	rdf:subject	<http://www.wikidata.org/entity/Q162990>
Blank node	rdf:predicate	bbkg:plays_for
Blank node	rdf:object	<http://www.wikidata.org/entity/Q36159>
Blank node	bbkg:start	"2010-01-01T00:00:00+00:00" xsd:dateTime
Blank node	bbkg:end	"2003-01-01T00:00:00+00:00" xsd:dateTime

5 External Source

RealGM was used as an external source to extend my knowledge graph with more data of NBA (The National Basketball Association) Players.
I have the RealGM ID (Number) assigned for each player from Wikidata and from it I was able to have the URL for each player in RealGM and link Information about players from both sources Wikidata and RealGM.

5.1 Metadata

The Information whether a player is active or inactive in not available in Wikidata. In RealGm each player's profile there is Current NBA Status field which includes whether a players is Under Contract (Team Name) or Unrestricted Free Agent.
This field was scraped to get the information for each player and extend our knowledge graph with whether a player is active or inactive.

```
<http://www.wikidata.org/entity/Q10862660> a bbkg:Active_Player,
        bbkg:Player ;
    bbkg:ID <https://basketball.realgm.com/player/Dennis-Schröder/summ
ary/26503> ;
    bbkg:contract "Under Contract (Los Angeles Lakers)" ;
    bbkg:full_name "Dennis Schröder" ;
```

Figure 8: Active_Player

```
<http://www.wikidata.org/entity/Q508026> a bbkg:Inactive_Player,
        bbkg:Player ;
    bbkg:ID <https://basketball.realgm.com/player/Andrew-Toney/summary/4665> ;
    bbkg:contract "Unrestricted Free Agent" ;
    bbkg:full_name "Andrew Toney" ;
    bbkg:participated_in <http://www.wikidata.org/entity/Q1150801> ;
    bbkg:play_as <http://www.wikidata.org/entity/Q273199> .
```

Figure 9: Inactive_Player

5.2 Temporal data

In RealGM there are Tables for each Award in Basketball by seasons.
Each award has the list of players that won this Award with timestamp and whether this player belongs to the 'East' or 'West' Conference. Figure 6: shows how the awards are in RealGM. I scraped this data for **NBA Player of the month**, **NBA Player of the week** and **NBA Rookie of the month**. Figure 7 below shows how the data looked like after scrapping it and adding it to the knowledge graph. Using Reification I extended it with conference and time. conference: Class and time: which has the temporal data of type timestamp.

2022-2023 NBA Player Of The Week

SORT: **PLAYER (A-Z)** COLUMNS: **SWIPE**

Player	Team	Conference	Date
Giannis Antetokounmpo	Milwaukee Bucks	East	Jan 30, 2023
Damian Lillard	Portland Trail Blazers	West	Jan 30, 2023
Jrue Holiday	Milwaukee Bucks	East	Jan 23, 2023
LeBron James	Los Angeles Lakers	West	Jan 23, 2023
Jalen Brunson	New York Knicks	East	Jan 16, 2023
Domantas Sabonis	Sacramento Kings	West	Jan 16, 2023
Donovan Mitchell	Cleveland Cavaliers	East	Jan 9, 2023
LeBron James	Los Angeles Lakers	West	Jan 9, 2023

Figure 10: NBA Player of the week table from RealGM

```
[] a rdf:Statement ;
    bbkg:conference bbkg:West ;
    bbkg:time "2011-04-01T00:00:00"^^xsd:dateTime ;
    rdf:object bbkg:NBA_Player_Of_The_Month ;
    rdf:predicate bbkg:is_awarded ;
    rdf:subject <http://www.wikidata.org/entity/Q29545> .
```

Figure 11: Snapshot of the award in bbkg

6 Interactive Prototype

In this Section I will talk about the questions that my implementation was able to find answers to.
1. List all the Players that plays in the same Position as "Brandon Hunter" ? Here i get the Position of the player Brandon Hunter and then retrieve all the players that play in the same position.
2. What is the Country, League, Division and Conference of the Sports_Club that "LeBron James" Plays_for? Here it retrieves all the information of the Sport_Club a Player plays for, those information includes League, Country and Division.
3. What is the status (contract) of the Player "LeBron James" and "Johnny Rogers" ? Here it gets the contract and accordingly it is added whether he is an Active or Inactive_Player
4. Who won the "Player of the month" Award in "2023.02.01", "Rookie of the month" Award in "2023.01.01"? Here it retrieves information about the awards and can filter by timestamp or by Conference (East or West).
5. What are the events that “Kobe Bryant” participated_in? Here it retrieves the event a Player participated in.

The Scrapping of awards or contract are on a Separate .py file that I re-run it and by that I have the Information on which I can extend my knowledge graph with.

References

[1] cellfie-plugin, 2023.

[2] Mappingmasterdsl, 2023.

[3] Henrik Dibowski and Stefan Schmid. Using knowledge graphs to manage a data lake. *INFORMATIK 2020*, 2021.

[4] Jacob Nilsson. *System of systems interoperability machine learning model*. PhD thesis, Luleå University of Technology, 2019.

[5] Bess Schrader. What's the difference between an ontology and a knowledge graph?, 2020.

[6] Huiqun Zhao, Shikan Zhang, and Junbao Zhao. Research of using protege to build ontology. In *2012 IEEE/ACIS 11th International Conference on Computer and Information Science*, pages 697–700. IEEE, 2012.